ISW036 - LABORATÓRIO DE DESENVOLVIMENTO PARA DISPOSITIVOS MÓVEIS

ATIVIDADE PRÁTICA - VERSÃO 0.2 DO PROJETO ANDROID

Prof. Dr. Desiré Nguessan

1. Nome dos Integrantes

1. Donizeti Almeida

2. Fabrício Andrade

3. Nikolas Nagy

4. Tiago Alexandre

2. Objetivo da atividade

Dar continuidade à Versão 0.1 do projeto, transformando o MVP preliminar em um primeiro incremento executável no Android Studio. Nesta etapa, a equipe deverá relacionar o fluxo principal às telas do aplicativo, criar a estrutura inicial do projeto e implementar uma parte do fluxo utilizando Kotlin.

VERSÃO 0.2 - PRIMEIRO INCREMENTO DO MVP ANDROID

Esta versão continua sendo preliminar. Como o briefing e a validação com o stakeholder poderão alterar requisitos e regras de negócio, a equipe deverá trabalhar com o MVP definido na Versão 0.1 sem tratar hipóteses ainda não validadas como fatos definitivos.

3. Revisão da Versão 0.1 após feedback do professor

Antes de iniciar a implementação, a equipe deverá revisar a Versão 0.1 considerando as observações recebidas e a discussão realizada com o professor. Os erros identificados deverão ser corrigidos e o MVP ajustado antes de ser utilizado como referência para o desenvolvimento.

Principais ajustes realizados após o feedback:

1. Correção da duplicação do requisito funcional referente às notificações locais, anteriormente listados como RF05 e RF06, RF06 excluida.

2. Correção da numeração dos Requisitos Não Funcionais (RNFs) para seguir a sequência correta (RNF01 e RNF02).

3. Remoção das notificações locais (US05) das funcionalidades essenciais do MVP, uma vez que foram priorizadas como "Should Have".

4. Confirmação de que a exportação de relatórios em PDF (US06) ficará fora do escopo do primeiro MVP, classificada como "Could Have".

5. Foco estrito no fluxo principal de auditoria, garantindo que os requisitos essenciais ("Must Have") sejam a base do primeiro incremento.

MVP revisado após feedback do professor: ☒ Sim ☐ Necessita novos ajustes

Tema do projeto: Auditoria Fiscal.

Problema preliminar: Auditores e equipes de controladoria costumam conduzir verificações fiscais com planilhas, papel e checklists soltos. Isso dificulta padronizar o registro de evidências, identificar não conformidades e acompanhar os apontamentos até a resolução.

Usuário principal presumido: Auditor fiscal ou auditor interno responsável pela execução da auditoria em campo.

Fluxo principal do MVP: Iniciar nova auditoria → Selecionar categoria fiscal → Verificar item do checklist → Marcar como conforme ou não conforme → Fotografar evidência → Registrar não conformidade (severidade, observação, prazo) → Finalizar auditoria → Visualizar resumo dos apontamentos.

Funcionalidades essenciais do MVP (3 a 5):

1. Checklist de auditoria organizado por categoria fiscal.
2. Captura de fotos como evidência, vinculadas a cada item.
3. Registro de não conformidades com severidade, observação e prazo.
4. Resumo da auditoria com apontamentos pendentes, em andamento e resolvidos.

4. Do fluxo principal para as telas

Identifiquem somente as telas necessárias para representar o fluxo principal do MVP. Evitem acrescentar telas ou funcionalidades que não sejam necessárias nesta etapa.

Ordem	Tela	Objetivo da tela	História(s) de usuário relacionada(s)
1	Tela de Home	Exibir o painel de gestão, proteger a edição por PIN e iniciar novas sessões.	US01
2	Tela de Categorias	Selecionar a categoria fiscal para iniciar a auditoria.	US01
3	Tela de Checklist	Exibir os itens a verificar e permitir marcar como conforme/não conforme.	US01
4	Tela da Câmera	Capturar fotos para servirem de evidência no local.	US02
5	Tela de Registro	Registrar a não conformidade com nível de severidade, observação e prazo.	US03
6	Tela de Resumo	Mostrar a visão consolidada com o status dos apontamentos.	US04

5. Histórias de usuário selecionadas para implementação

Selecionem as histórias da Versão 0.1 que serão contempladas neste primeiro incremento. Não é necessário implementar todos os requisitos funcionais levantados.

US01: Como auditor, quero percorrer um checklist organizado por categoria fiscal, para verificar todos os itens da auditoria de forma padronizada.

US02: Como auditor, quero fotografar evidências e vinculá-las ao item verificado, para comprovar o que foi constatado em campo.

US03: Como auditor, quero registrar uma não conformidade com severidade e observação, para documentar os problemas encontrados.

US04: Como auditor, quero visualizar um resumo da auditoria com o status dos apontamentos, para saber o que está pendente, em andamento ou resolvido.

6. Estrutura inicial do projeto Android

Criem o projeto no Android Studio e registrem as principais definições:

Nome do aplicativo/projeto: Auditoria Fiscal / auditoriafiscal

Package: com.example.auditoriafiscal

Tela inicial: TelaHome (Configurada via NavHost dentro da MainActivity.kt, para abrigar o painel de gestão com abas e o controle de acesso por PIN).

Outras telas implementadas: TelaChecklistAuditoria (lista de itens) e o componente de interface DialogNaoConformidade (para o registro do apontamento).

Requisitos mínimos desta etapa:

- Projeto criado e executando no Android Studio.
- Pelo menos 2 telas do fluxo principal implementadas.
- Navegação funcionando entre as telas implementadas.
- Uso de dados fixos ou simulados quando necessário.
- Não é obrigatório implementar Firebase, banco de dados ou todos os RFs nesta atividade.

7. Classe Kotlin do domínio do projeto

Criem pelo menos uma classe Kotlin relacionada ao domínio do projeto. A classe deve representar uma entidade relevante do MVP.

Nome da classe: NaoConformidade

O que ela representa no projeto: Representa o registro formal de um problema ou irregularidade encontrada durante a inspeção de um item do checklist de auditoria.

Propriedades principais:

1. idItem (Int)
2. nivelSeveridade (String)
3. observacao (String)
4. prazoResolucao (String)
5. status (String)

Métodos/comportamentos, se aplicável:

1. adicionarEvidencia(foto: ByteArray)
2. obterStatus(): String

8. Primeiro incremento do MVP

Descrevam o que efetivamente foi implementado e pode ser demonstrado no aplicativo ao final da atividade.

Item	Implementado?	Observação
Projeto executa no dispositivo/emulador	☒ Sim ☐ Não	A aplicação compila e roda no emulador Android utilizando Jetpack Compose.
Tela 1 do fluxo principal	☒ Sim ☐ Não	TelaCategorias exibe a lista inicial de categorias fiscais para seleção.
Tela 2 do fluxo principal	☒ Sim ☐ Não	TelaChecklistAuditoria exibe os itens vinculados à categoria escolhida.
Navegação entre as telas	☒ Sim ☐ Não	Implementada com sucesso utilizando a biblioteca Navigation Compose (NavController).
Classe Kotlin do domínio	☒ Sim ☐ Não	A data class NaoConformidade é criada ao salvar um apontamento no Dialog.
Dados simulados utilizados no fluxo	☒ Sim ☐ Não	As listas de categorias e os itens de auditoria estão mockados diretamente no código.

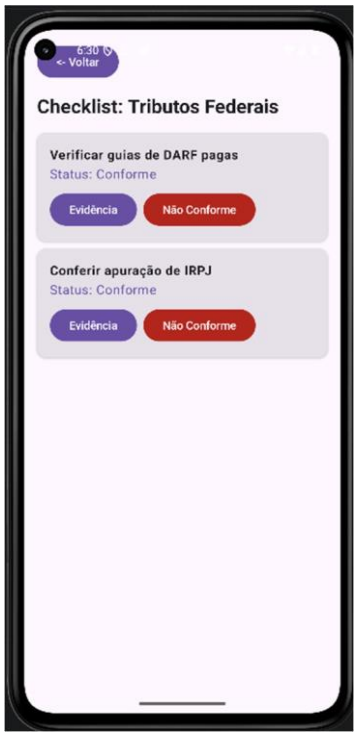
9. Evidências da execução

Incluam no documento pelo menos 2 capturas de tela do aplicativo em execução, mostrando as telas e a navegação implementadas.

Evidência 1:



Evidência 2:



10. Pendências e hipóteses para validação

Registrem aspectos que ainda dependem do briefing com o stakeholder ou que não devem ser considerados definitivos nesta versão.

1. Definir com exatidão as categorias fiscais reais e os itens de checklist exigidos pelo stakeholder.

2. Confirmar os níveis exatos de severidade utilizados na empresa e como os prazos de resolução devem ser estabelecidos.

3. Validar a real necessidade e formato das evidências fotográficas (como serão armazenadas no dispositivo ou em nuvem futuramente).

4. O controle de acesso por PIN (1234) atual é estático e mockado no código, dependendo de validação futura com o stakeholder para integração com biometria ou backend de autenticação.

5. A captura de evidências salva o arquivo localmente via FileProvider, mas a persistência de longo prazo (nuvem) e a limpeza de cache automático deverão ser refinadas nas próximas versões

11. Checklist de entrega

☒ MVP da Versão 0.1 revisado.

☒ Fluxo principal identificado.

☒ Telas necessárias ao fluxo definidas.

☒ Histórias de usuário relacionadas às telas.

☒ Projeto criado e executando no Android Studio.

☒ Pelo menos 2 telas implementadas.

☒ Navegação funcionando entre as telas.

☒ Pelo menos 1 classe Kotlin do domínio criada.

☒ Dados simulados utilizados quando necessário.

☒ Pelo menos 2 evidências da execução inseridas.

☒ Pendências/hipóteses para validação registradas.

IMPORTANTE: o objetivo desta atividade não é concluir todo o aplicativo. A equipe deverá demonstrar um primeiro incremento funcional e coerente com o MVP preliminar. Requisitos e regras de negócio poderão ser revisados após o briefing com o stakeholder.

12. Apresentação e Avaliação:

Na próxima aula, cada equipe deverá realizar um **pitch de 10 a 15 minutos**, apresentando o trabalho desenvolvido nesta etapa e demonstrando o **primeiro incremento do aplicativo em funcionamento**. Todos os integrantes deverão estar preparados para participar da apresentação e responder às questões relacionadas ao projeto.

Caso alguma funcionalidade prevista não esteja funcionando, a equipe deverá demonstrar o que foi implementado, explicar objetivamente a pendência e apresentar o que falta para concluí-la.